

```

1 //#####
2 // FILE:   FinalProjectStarter_main.c
3 //
4 // TITLE:   Final Project Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "F2837xD_SWPrioritizedIsrLevels.h"
16 #include "driverlib.h"
17 #include "device.h"
18 #include "F28379dSerial.h"
19 #include "song.h"
20 #include "dsp.h"
21 #include "fpu32/fpu_rfft.h"
22 #include "xy.h"
23 #include "MatrixMath.h"
24 #include "SE423Lib.h"
25 #include "OptiTrack.h"
26
27 #define PI          3.1415926535897932384626433832795
28 #define TWOPI       6.283185307179586476925286766559
29 #define HALFPI      1.5707963267948966192313216916398
30 // The Launchpad's CPU Frequency set to 200 you should not change this value
31 #define LAUNCHPAD_CPU_FREQUENCY 200
32
33 // ----- code for CAN start here -----
34 #include "F28379dCAN.h"
35 // #define TX_MSG_DATA_LENGTH      4
36 // #define TX_MSG_OBJ_ID          0 //transmit
37
38 #define RX_MSG_DATA_LENGTH      8
39 #define RX_MSG_OBJ_ID_1        1 //measurement from sensor 1
40 #define RX_MSG_OBJ_ID_2        2 //measurement from sensor 2
41 #define RX_MSG_OBJ_ID_3        3 //quality from sensor 1
42 #define RX_MSG_OBJ_ID_4        4 //quality from sensor 2
43 // ----- code for CAN end here -----
44
45 // Interrupt Service Routines predefinition
46 __interrupt void cpu_timer0_isr(void);
47 __interrupt void cpu_timer1_isr(void);
48 __interrupt void cpu_timer2_isr(void);
49 __interrupt void SWI1_HighestPriority(void);
50 __interrupt void SWI2_MiddlePriority(void);
51 __interrupt void SWI3_LowestPriority(void);
52 __interrupt void ADCC_ISR(void);
53 __interrupt void SPIB_isr(void);
54 // ----- code for CAN start here -----
55 __interrupt void can_isr(void);
56 // ----- code for CAN end here -----
57
58 void setF28027EPWM1A(float controleffort);
59 int16_t EPwm1A_F28027 = 1500;
60 void setF28027EPWM2A(float controleffort);
61 int16_t EPwm2A_F28027 = 1500;
62
63 // ----- code for CAN start here -----
64 // volatile uint32_t txMsgCount = 0;
65 // extern uint16_t txMsgData[4];
66
67 volatile uint32_t rxMsgCount_1 = 0;
68 volatile uint32_t rxMsgCount_2 = 0;
69 extern uint16_t rxMsgData[8];
70
71 uint32_t dis_raw_1[2];
72 uint32_t dis_raw_2[2];
73 uint32_t dis_1 = 0;
74 uint32_t dis_2 = 0;
75
76 uint32_t quality_raw_1[4];
77 uint32_t quality_raw_2[4];
78 float quality_1 = 0.0;
79 float quality_2 = 0.0;
80
81 uint32_t lightlevel_raw_1[4];
82 uint32_t lightlevel_raw_2[4];
83 float lightlevel_1 = 0.0;
84 float lightlevel_2 = 0.0;
85
86 uint32_t measure_status_1 = 0;
87 uint32_t measure_status_2 = 0;
88
89 volatile uint32_t errorFlag = 0;
90 // ----- code for CAN end here -----
91
92 uint32_t numTimer0calls = 0;
93 uint16_t UARTPrint = 0;
94
95 float printLV1 = 0;
96 float printLV2 = 0;
97
98 float printLinux1 = 0;
99 float printLinux2 = 0;
100
101 uint16_t LADARi = 0;
102 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
103 uint16_t G_len = 11; //length of command
104 xy_ladar_pts[228]; //xy data
105 float LADARrightfront = 0;
106 float LADARfront = 0;
107 float LADARtemp_x = 0;
108 float LADARtemp_y = 0;
109 extern datapts_ladar_data[228];
110
111 extern uint16_t newLinuxCommands;
112 extern float LinuxCommands[CMDNUM_FROM_FLOATS];

```

```

113
114 extern uint16_t NewLVData;
115 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
116 extern LVSendFloats_t DataToLabView;
117 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
118 extern uint16_t LADARpingpong;
119 extern uint16_t NewCAMDataThreshold1; // Flag new data
120 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
121 extern uint16_t NewCAMDataThreshold2; // Flag new data
122 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
123
124 extern uint16_t NewCAMDataAprilTag1; // Flag new data
125 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
126
127 float tagid = 0;
128 float tagx = 0;
129 float tagy = 0;
130 float tagz = 0;
131 float tagthetax = 0;
132 float tagthetay = 0;
133 float tagthetaz = 0;
134 int32_t numtag = 0;
135 int32_t AprilTagState = 10;
136 float KpAprilTag = -1.0;
137 float AprilTagSetPoint = 0;
138 int32_t AprilTagCount = 0;
139
140 float MaxAreaThreshold1 = 0;
141 float MaxColThreshold1 = 0;
142 float MaxRowThreshold1 = 0;
143 float NextLargestAreaThreshold1 = 0;
144 float NextLargestColThreshold1 = 0;
145 float NextLargestRowThreshold1 = 0;
146 float NextNextLargestAreaThreshold1 = 0;
147 float NextNextLargestColThreshold1 = 0;
148 float NextNextLargestRowThreshold1 = 0;
149
150 float MaxAreaThreshold2 = 0;
151 float MaxColThreshold2 = 0;
152 float MaxRowThreshold2 = 0;
153 float NextLargestAreaThreshold2 = 0;
154 float NextLargestColThreshold2 = 0;
155 float NextLargestRowThreshold2 = 0;
156 float NextNextLargestAreaThreshold2 = 0;
157 float NextNextLargestColThreshold2 = 0;
158 float NextNextLargestRowThreshold2 = 0;
159
160 uint32_t numThres1 = 0;
161 uint32_t numThres2 = 0;
162
163 pose ROBOTps = {0,0,0}; //robot position
164 pose LADARps = {3.5/12,0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
165 float LADARxoffset = 0;
166 float LADARyoffset = 0;
167
168 uint32_t timecount = 0;
169 int16_t RobotState = 1;
170 int16_t checkfronttally = 0;
171 int32_t WallFollowtime = 0;
172
173 #define NUMWAYPOINTS 10
174 uint16_t statePos = 0;
175 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
176 uint16_t i = 0; //for loop
177
178 uint16_t right_wall_follow_state = 2; // right follow
179 float Kp_front_wall = -2.0;
180 float front_turn_velocity = 0.2;
181 float left_turn_Stop_threshold = 3.5;
182 float Kp_right_wal = -4.0;
183 float ref_right_wall = 1.1;
184 float foward_velocity = 1.0;
185 float left_turn_Start_threshold = 1.3;
186 float turn_saturation = 2.5;
187
188 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
189
190 //more kalman vars
191 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
192 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
193 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
194 float z[3][1]; // state measurement
195 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
196 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
197 #define ProcUncert 0.0001
198 #define CovScalar 10
199 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
200 {0,ProcUncert,ProcUncert/CovScalar},
201 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
202 #define MeasUncert 1
203 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
204 {0,MeasUncert,MeasUncert/CovScalar},
205 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of OptiTrack Motion Capture measurement)
206 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
207 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
208 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
209 float temp_3x3[3][3]; // intermediate storage
210 float temp_3x1[3][1]; // intermediate storage
211 float ytilde[3][1]; // difference between predictions
212
213 // Optitrack Variables
214 int32_t OptiNumUpdatesProcessed = 0;
215 pose OPTITRACKps;
216 extern uint16_t new_optitrack;
217 extern float Optitrackdata[OPTITRACKDATASIZE];
218 int16_t newOPTITRACKpose=0;
219
220 int16_t adcc2result = 0;
221 int16_t adcc3result = 0;
222 int16_t adcc4result = 0;
223 int16_t adcc5result = 0;
224 float adcc2Volt = 0.0;
225 float adcc3Volt = 0.0;
226 float adcc4Volt = 0.0;

```

```

227 float adcC5Volt = 0.0;
228 int32_t numADCCalls = 0;
229 float LeftWheel = 0;
230 float RightWheel = 0;
231 float LeftWheel_1 = 0;
232 float RightWheel_1 = 0;
233 float LeftVel = 0;
234 float RightVel = 0;
235 float uLeft = 0;
236 float uRight = 0;
237 float HandValue = 0;
238 float vref = 0;
239 float turn = 0;
240 float gyro9250_drift = 0;
241 float gyro9250_angle = 0;
242 float old_gyro9250 = 0;
243 float gyroLPR510_angle = 0;
244 float gyroLPR510_offset = 0;
245 float gyroLPR510_drift = 0;
246 float old_gyroLPR510 = 0;
247 float gyro9250_radians = 0;
248 float gyroLPR510_radians = 0;
249
250 int16_t readdata[25];
251 int16_t IMU_data[9];
252
253 float accelx = 0;
254 float accely = 0;
255 float accelz = 0;
256 float gyrox = 0;
257 float gyroy = 0;
258 float gyroz = 0;
259
260 // Needed global Variables
261 float accelx_offset = 0;
262 float accely_offset = 0;
263 float accelz_offset = 0;
264 float gyrox_offset = 0;
265 float gyroy_offset = 0;
266 float gyroz_offset = 0;
267 int16_t calibration_state = 0;
268 int32_t calibration_count = 0;
269 int16_t doneCal = 0;
270
271 #define MPU9250 1
272 #define DAN28027 2
273 int16_t CurrentChip = MPU9250;
274 int16_t DAN28027Garbage = 0;
275 int16_t dan28027adc1 = 0;
276 int16_t dan28027adc2 = 0;
277 uint16_t MPU9250ignoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
278
279 float RCangle;
280
281 void main(void)
282 {
283     // PLL, WatchDog, enable Peripheral Clocks
284     // This example function is found in the F2837xD_SysCtrl.c file.
285     InitSysCtrl();
286
287     InitGpio();
288
289     InitSE423DefaultGPIO();
290
291     //Scope for Timing
292     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
293     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSHPULL);
294     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
295
296     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
297     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
298     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
299
300     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
301     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSHPULL);
302     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
303
304     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
305     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
306     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
307
308     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
309     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
310     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
311
312     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
313     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSHPULL);
314     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
315
316 // ----- code for CAN start here -----
317 //GPIO17 - CANRXB
318 GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
319 GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
320
321 //GPIO12 - CANTXB
322 GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
323 GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSHPULL);
324 // ----- code for CAN end here -----
325
326
327
328 // ----- code for CAN start here -----
329 // Initialize the CAN controller
330 InitCANB();
331
332 // Set up the CAN bus bit rate to 1000 kbps
333 setCANBitRate(200000000, 1000000);
334
335 // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
336 CanbRegs.CAN_CTL.bit.IE0= 1;
337 CanbRegs.CAN_CTL.bit.EIE= 1;
338 // ----- code for CAN end here -----
339
340 // Clear all interrupts and initialize PIE vector table:

```

```

341 // Disable CPU interrupts
342 DINT;
343
344 // Initialize the PIE control registers to their default state.
345 // The default state is all PIE interrupts disabled and flags
346 // are cleared.
347 // This function is found in the F2837xD_PieCtrl.c file.
348 InitPieCtrl();
349
350 // Disable CPU interrupts and clear all CPU interrupt flags:
351 IER = 0x0000;
352 IFR = 0x0000;
353
354 // Initialize the PIE vector table with pointers to the shell Interrupt
355 // Service Routines (ISR).
356 // This will populate the entire table, even if the interrupt
357 // is not used in this example. This is useful for debug purposes.
358 // The shell ISR routines are found in F2837xD_DefaultIsr.c.
359 // This function is found in F2837xD_PieVect.c.
360 InitPieVectTable();
361
362 // Interrupts that are used in this example are re-mapped to
363 // ISR functions found within this project
364 EALLOW; // This is needed to write to EALLOW protected registers
365 PieVectTable.TIMER0_INT = &cpu_timer0_isr;
366 PieVectTable.TIMER1_INT = &cpu_timer1_isr;
367 PieVectTable.TIMER2_INT = &cpu_timer2_isr;
368 PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
369 PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
370 PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
371 PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
372 PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
373 PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
374 PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
375 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
376 PieVectTable.ADCCL_INT = &ADCC_ISR;
377 PieVectTable.SPIB_RX_INT = &SPIB_isr;
378 PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
379 PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
380 PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
381 // ----- code for CAN start here -----
382 PieVectTable.CANB0_INT = &can_isr;
383 // ----- code for CAN end here -----
384 EDIS; // This is needed to disable write to EALLOW protected registers
385
386 // Initialize the CpuTimers Device Peripheral. This function can be
387 // found in F2837xD_CpuTimers.c
388 InitCpuTimers();
389
390 // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
391 // 200MHz CPU Freq, Period (in uSeconds)
392 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
393 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to command LADAR every 100ms. Do not Change.
394 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
395
396 // Enable CpuTimer Interrupt bit TIE
397 CpuTimer0Regs.TCR.all = 0x4000;
398 CpuTimer1Regs.TCR.all = 0x4000;
399 CpuTimer2Regs.TCR.all = 0x4000;
400
401 DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
402
403 init_serialSCIB(&SerialB,19200);
404 init_serialSCIC(&SerialC,19200);
405
406 for (LADARi = 0; LADARi < 228; LADARi++) {
407     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
408 }
409
410 init_eQEPs();
411 init_EPWM1and2();
412 init_RCServoPWM_3AB_5AB_6A();
413 init_ADCsAndDACs();
414 setupSpib();
415
416 EALLOW;
417 EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
418 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
419 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
420 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
421 EPwm4Regs.TBCTR = 0x0; // Clear counter
422 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
423 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
424 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
425 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
426 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
427 EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
428 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
429 EDIS;
430
431 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
432 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
433 // to CPU-Timer 2: int 12 is for the SWI.
434 IER |= M_INT1;
435 IER |= M_INT6;
436 IER |= M_INT8; // SCIC SCID
437 IER |= M_INT9; // SCIA
438 IER |= M_INT12;
439 IER |= M_INT13;
440 IER |= M_INT14;
441
442 // Enable TINT0 in the PIE: Group 1 interrupt 7
443 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
444 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
445 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
446 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
447 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
448 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
449 // ----- code for CAN start here -----
450 // Enable CANB in the PIE: Group 9 interrupt 7
451 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
452 // ----- code for CAN end here -----
453
454 // ----- code for CAN start here -----

```

```

455 // Enable the CAN interrupt signal
456 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
457 // ----- code for CAN end here -----
458
459
460 robotdest[0].x = -4;    robotdest[0].y = 10;
461 robotdest[1].x = -4;    robotdest[1].y = 2;
462 //middle of bottom
463 robotdest[2].x = 0;     robotdest[2].y = 2;
464 //outside the course
465 robotdest[3].x = 0;     robotdest[3].y = -3;
466 //back to middle
467 robotdest[4].x = 0;     robotdest[4].y = 2;
468 robotdest[5].x = 4;     robotdest[5].y = 2;
469 robotdest[6].x = 4;     robotdest[6].y = 10;
470 robotdest[7].x = 0;     robotdest[7].y = 9;
471 robotdest[8].x = 4;     robotdest[8].y = 5;
472 robotdest[9].x = 2;     robotdest[9].y = 8;
473
474 // ROBOTps will be updated by Optitrack during gyro calibration
475 // TODO: specify the starting position of the robot
476 ROBOTps.x = 0;          //the estimate in array form (useful for matrix operations)
477 ROBOTps.y = 0;
478 ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
479 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
480 x_pred[1][0] = ROBOTps.y;
481 x_pred[2][0] = ROBOTps.theta;
482
483 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
484 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
485 SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
486 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
487 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
488 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
489
490 init_serialSCIA(&SerialA,115200);
491 init_serialSCID(&SerialD,2083332);
492
493 // Enable global Interrupts and higher priority real-time debug events
494 EINT; // Enable Global interrupt INTM
495 ERTM; // Enable Global realtime interrupt DBGM
496 // ----- code for CAN start here -----
497 // Measured Distance from 1
498 // Initialize the receive message object 1 used for receiving CAN messages.
499 // Message Object Parameters:
500 //     Message Object ID Number: 1
501 //     Message Identifier: 0x060b0101
502 //     Message Frame: Standard
503 //     Message Type: Receive
504 //     Message ID Mask: 0x0
505 //     Message Object Flags: Receive Interrupt
506 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
507 //     for a Receive mailbox)
508 //
509 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
510                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
511                       RX_MSG_DATA_LENGTH);
512
513 // Measured Distance from 2
514 // Initialize the receive message object 2 used for receiving CAN messages.
515 // Message Object Parameters:
516 //     Message Object ID Number: 2
517 //     Message Identifier: 0x060b0102
518 //     Message Frame: Standard
519 //     Message Type: Receive
520 //     Message ID Mask: 0x0
521 //     Message Object Flags: Receive Interrupt
522 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
523 //     for a Receive mailbox)
524 //
525
526 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
527                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
528                       RX_MSG_DATA_LENGTH);
529
530 // Measurement Quality from 1
531 // Initialize the receive message object 2 used for receiving CAN messages.
532 // Message Object Parameters:
533 //     Message Object ID Number: 3
534 //     Message Identifier: 0x060b0201
535 //     Message Frame: Standard
536 //     Message Type: Receive
537 //     Message ID Mask: 0x0
538 //     Message Object Flags: Receive Interrupt
539 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
540 //     for a Receive mailbox)
541 //
542
543 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
544                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
545                       RX_MSG_DATA_LENGTH);
546
547 // Measurement Quality from 2
548 // Initialize the receive message object 2 used for receiving CAN messages.
549 // Message Object Parameters:
550 //     Message Object ID Number: 4
551 //     Message Identifier: 0x060b0202
552 //     Message Frame: Standard
553 //     Message Type: Receive
554 //     Message ID Mask: 0x0
555 //     Message Object Flags: Receive Interrupt
556 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
557 //     for a Receive mailbox)
558 //
559
560 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
561                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
562                       RX_MSG_DATA_LENGTH);
563
564 //
565 // Start CAN module operations
566 //
567 CanbRegs.CAN_CTL.bit.Init = 0;
568 CanbRegs.CAN_CTL.bit.CCE = 0;

```

```

569
570
571 // ----- code for CAN end here -----
572 char S_command[19] = "S1152000124000\n"; //this change the baud rate to 115200
573 uint16_t S_len = 19;
574 serial_sendSCIC(&SerialC, S_command, S_len);
575
576
577 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
578 init_serialSCIC(&SerialC, 115200);
579
580
581 // LED1 is GPIO22
582 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
583 // LED2 is GPIO94
584 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
585 // LED3 is GPIO95
586 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
587 // LED4 is GPIO97
588 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
589 // LED5 is GPIO111
590 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
591
592
593 // IDLE loop. Just sit and loop forever (optional):
594 while(1)
595 {
596     if (UARTPrint == 1 ) {
597 /*
598         if (readbuttons() == 0) {
599             UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
600             UART_printfLine(1, "x:%.2f y:%.2f a:%.2f", ROBOTps.x, ROBOTps.y, ROBOTps.theta);
601             UART_printfLine(2, "F%.4f R%.4f", LADARfront, LADARrightfront);
602         } else if (readbuttons() == 1) {
603             UART_printfLine(1, "O1A:%.0fC:%.0fR:%.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
604             UART_printfLine(2, "P1A:%.0fC:%.0fR:%.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
605             //UART_printfLine(1, "LV1:%.3f LV2:%.3f", printLV1, printLV2);
606             //UART_printfLine(2, "Ln1:%.3f Ln2:%.3f", printLinux1, printLinux2);
607         } else if (readbuttons() == 2) {
608             UART_printfLine(1, "O2A:%.0fC:%.0fR:%.0f", NextLargestAreaThreshold1, NextLargestColThreshold1, NextLargestRowThreshold1);
609             UART_printfLine(2, "P2A:%.0fC:%.0fR:%.0f", NextLargestAreaThreshold2, NextLargestColThreshold2, NextLargestRowThreshold2);
610             // UART_printfLine(1, "%.2f %.2f", adcC2Volt, adcC3Volt);
611             // UART_printfLine(2, "%.2f %.2f", adcC4Volt, adcC5Volt);
612         } else if (readbuttons() == 4) {
613             UART_printfLine(1, "O3A:%.0fC:%.0fR:%.0f", NextNextLargestAreaThreshold1, NextNextLargestColThreshold1, NextNextLargestRowThreshold1);
614             UART_printfLine(2, "P3A:%.0fC:%.0fR:%.0f", NextNextLargestAreaThreshold2, NextNextLargestColThreshold2, NextNextLargestRowThreshold2);
615             // UART_printfLine(1, "L:%.3f R:%.3f", LeftVel, RightVel);
616             // UART_printfLine(2, "uL:%.2f uR:%.2f", uLeft, uRight);
617         } else if (readbuttons() == 8) {
618             UART_printfLine(1, "020x%.2f y%.2f", ladar_pts[20].x, ladar_pts[20].y);
619             UART_printfLine(2, "150x%.2f y%.2f", ladar_pts[150].x, ladar_pts[150].y);
620         } else if (readbuttons() == 3) {
621             UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
622             UART_printfLine(2, "MPU:%.2f LPR:%.2f", gyro9250_radians, gyroLPR510_radians);
623         } else if (readbuttons() == 5) {
624             UART_printfLine(1, "0x:%.2f 0y:%.2f 0a:%.2f", OPTITRACKps.x, OPTITRACKps.y, OPTITRACKps.theta);
625             UART_printfLine(2, "State:%d : %d", RobotState, statePos);
626         } else if (readbuttons() == 6) {
627             UART_printfLine(1, "D1 %ld D2 %ld", dis_1, dis_2);
628             UART_printfLine(2, "St1 %ld St2 %ld", measure_status_1, measure_status_2);
629         } else if (readbuttons() == 7) {
630             UART_printfLine(1, "%.0f, %.1f, %.1f, %.1f", tagid, tagx, tagy, tagz);
631             UART_printfLine(2, "%.1f, %.1f, %.1f", tagthetax, tagthetay, tagthetaz);
632         }
633         UART_printfLine(1, "RCangle %.1f ", RCangle);
634
635         UARTPrint = 0;
636     }
637 }
638 }
639
640 __interrupt void SPIB_isr(void){
641
642     uint16_t i;
643     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
644
645     GpioDataRegs.GPCSET.bit.GPIO66 = 1; //Pull CS high as done R/W
646     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
647
648     if (CurrentChip == MPU9250) {
649
650         for (i=0; i<8; i++) {
651             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
652         }
653
654         PostSWI1(); // Manually cause the interrupt for the SWI1
655
656     } else if (CurrentChip == DAN28027) {
657
658         DAN28027Garbage = SpibRegs.SPIRXBUF;
659         dan28027adc1 = SpibRegs.SPIRXBUF;
660         dan28027adc2 = SpibRegs.SPIRXBUF;
661         CurrentChip = MPU9250;
662         SpibRegs.SPIFFCT.bit.TXDLY = 0;
663     }
664
665     SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
666     SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
667     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
668     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
669 }
670
671
672 //adcd1 pie interrupt
673 __interrupt void ADCC_ISR (void)
674 {
675     GpioDataRegs.GPASET.bit.GPIO11 = 1;
676     adcc2result = AdccResultRegs.ADCRESULT0;
677     adcc3result = AdccResultRegs.ADCRESULT1;
678     adcc4result = AdccResultRegs.ADCRESULT2;
679     adcc5result = AdccResultRegs.ADCRESULT3;
680
681     // Here covert ADCIND0 to volts
682     adcC2Volt = adcc2result*3.0/4095.0;

```

```

683     adcC3Volt = adcc3result*3.0/4095.0;
684     adcC4Volt = adcc4result*3.0/4095.0;
685     adcC5Volt = adcc5result*3.0/4095.0;
686
687     if (MPU9250ignoreCNT >= 1) {
688         CurrentChip = MPU9250;
689         SpibRegs.SPIFFCT.bit.TXDLY = 0;
690         SpibRegs.SPIFFRX.bit.RXFFIL = 8;
691
692         GpioDataRegs.GPCLEAR.bit.GPIO66 = 1;
693
694         SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
695         SpibRegs.SPITXBUF = 0;
696         SpibRegs.SPITXBUF = 0;
697         SpibRegs.SPITXBUF = 0;
698         SpibRegs.SPITXBUF = 0;
699         SpibRegs.SPITXBUF = 0;
700         SpibRegs.SPITXBUF = 0;
701         SpibRegs.SPITXBUF = 0;
702     } else {
703         MPU9250ignoreCNT++;
704     }
705
706     numADCCalls++;
707     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
708     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
709     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
710 }
711 }
712
713 // cpu_timer0_isr - CPU Timer0 ISR
714 __interrupt void cpu_timer0_isr(void)
715 {
716     CpuTimer0.InterruptCount++;
717
718     numTimer0calls++;
719
720     if ((numTimer0calls%5) == 0) {
721         // Blink LaunchPad Red LED
722         //GpioDataRegs.GPBTGGLE.bit.GPIO34 = 1;
723     }
724
725     // Acknowledge this interrupt to receive more interrupts from group 1
726     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
727 }
728
729 // cpu_timer1_isr - CPU Timer1 ISR
730 __interrupt void cpu_timer1_isr(void)
731 {
732
733     serial_sendSCIC(&SerialC, G_command, G_len);
734
735     CpuTimer1.InterruptCount++;
736 }
737
738 // cpu_timer2_isr CPU Timer2 ISR
739 __interrupt void cpu_timer2_isr(void)
740 {
741     // Blink LaunchPad Blue LED
742     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
743
744     CpuTimer2.InterruptCount++;
745
746     PostSWI3();
747
748
749     // if ((CpuTimer2.InterruptCount % 10) == 0) {
750     //     UARTPrint = 1;
751     // }
752 }
753
754 void setF28027EPWM1A(float controleffort){
755     if (controleffort < -10) {
756         controleffort = -10;
757     }
758     if (controleffort > 10) {
759         controleffort = 10;
760     }
761     float value = (controleffort+10)*3000.0/20.0;
762     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
763 }
764 void setF28027EPWM2A(float controleffort){
765     if (controleffort < -10) {
766         controleffort = -10;
767     }
768     if (controleffort > 10) {
769         controleffort = 10;
770     }
771     float value = (controleffort+10)*3000.0/20.0;
772     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
773 }
774
775 //
776 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
777 //
778 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
779 {
780     GpioDataRegs.GPASET.bit.GPIO61 = 1;
781     // Set interrupt priority:
782     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
783     IER |= M_INT12;
784     IER &= MINT12; // Set "global" priority
785     PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
786     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
787     __asm(" NOP");
788     EINT;
789
790     //#####
791     IMU_data[0] = readdata[1];
792     IMU_data[1] = readdata[2];
793     IMU_data[2] = readdata[3];
794     IMU_data[3] = readdata[5];
795     IMU_data[4] = readdata[6];
796     IMU_data[5] = readdata[7];

```

```

797
798 accelx = (((float)(IMU_data[0]))*4.0/32767.0);
799 accely = (((float)(IMU_data[1]))*4.0/32767.0);
800 accelz = (((float)(IMU_data[2]))*4.0/32767.0);
801 gyrox = (((float)(IMU_data[3]))*250.0/32767.0);
802 gyroy = (((float)(IMU_data[4]))*250.0/32767.0);
803 gyroz = (((float)(IMU_data[5]))*250.0/32767.0);
804
805 if(calibration_state == 0){
806     calibration_count++;
807     if (calibration_count == 2000) {
808         calibration_state = 1;
809         calibration_count = 0;
810     }
811 } else if(calibration_state == 1){
812     accelx_offset+=accelx;
813     accely_offset+=accely;
814     accelz_offset+=accelz;
815     gyrox_offset+=gyrox;
816     gyroy_offset+=gyroy;
817     gyroz_offset+=gyroz;
818     gyroLPR510_offset+=adcC5Volt;
819     calibration_count++;
820     if (calibration_count == 2000) {
821         calibration_state = 2;
822         accelx_offset/=2000.0;
823         accely_offset/=2000.0;
824         accelz_offset/=2000.0;
825         gyrox_offset/=2000.0;
826         gyroy_offset/=2000.0;
827         gyroz_offset/=2000.0;
828         gyroLPR510_offset/=2000.0;
829         calibration_count = 0;
830     }
831 } else if(calibration_state == 2) {
832
833     gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
834     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
835
836     gyro9250_drift += (((gyroz-gyroLPR510_offset) + old_gyro9250)*.0005)/(2000);
837     old_gyro9250 = gyroLPR510-gyroLPR510_offset;
838     calibration_count++;
839
840     if (calibration_count == 2000) {
841         calibration_state = 3;
842         calibration_count = 0;
843         doneCal = 1;
844         newOPTITRACKpose = 0;
845     }
846 } else if(calibration_state == 3){
847     accelx -=(accelx_offset);
848     accely -=(accely_offset);
849     accelz -=(accelz_offset);
850     gyrox -= gyrox_offset;
851     gyroy -= gyroy_offset;
852     gyroz -= gyroLPR510_offset;
853     LeftWheel = readEncLeft();
854     RightWheel = readEncRight();
855     HandValue = readEncWheel();
856
857     gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
858     old_gyro9250 = gyroLPR510;
859
860     gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
861     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
862
863     gyro9250_radians = (gyro9250_angle * (PI/180.0));
864     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
865
866     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
867     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
868
869     if (newLinuxCommands == 1) {
870         newLinuxCommands = 0;
871         printLinux1 = LinuxCommands[0];
872         printLinux2 = LinuxCommands[1];
873         /// = LinuxCommands[2];
874         /// = LinuxCommands[3];
875         /// = LinuxCommands[4];
876         /// = LinuxCommands[5];
877         /// = LinuxCommands[6];
878         /// = LinuxCommands[7];
879         /// = LinuxCommands[8];
880         /// = LinuxCommands[9];
881         /// = LinuxCommands[10];
882     }
883
884     if (NewCAMDataThreshold1 == 1) {
885         NewCAMDataThreshold1 = 0;
886         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
887         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
888         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
889
890         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
891         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
892         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
893
894         NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
895         NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
896         NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
897         numThres1++;
898         if ((numThres1 % 5) == 0) {
899             // LED4 is GPIO97
900             GpioDataRegs.GPDT0GGLE.bit.GPIO97 = 1;
901         }
902     }
903
904     if (NewCAMDataThreshold2 == 1) {
905         NewCAMDataThreshold2 = 0;
906         MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
907         MaxColThreshold2 = fromCAMvaluesThreshold2[1];
908         MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
909
910         NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];

```

```

911     NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
912     NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
913
914     NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
915     NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
916     NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
917     numThres2++;
918     if ((numThres2 % 5) == 0) {
919         // LED5 is GPIO111
920         GpioDataRegs.GPDT0GGLE.bit.GPIO111 = 1;
921     }
922 }
923
924 if (NewCAMDataAprilTag1 == 1) {
925     NewCAMDataAprilTag1 = 0;
926     tagx = fromCAMvaluesAprilTag1[0];
927     tagy = fromCAMvaluesAprilTag1[1];
928     tagz = fromCAMvaluesAprilTag1[2];
929
930     tagthetax = fromCAMvaluesAprilTag1[3];
931     tagthetay = fromCAMvaluesAprilTag1[4];
932     tagthetaz = fromCAMvaluesAprilTag1[5];
933
934     tagid = fromCAMvaluesAprilTag1[6];
935
936     numtag++;
937     if ((numtag % 5) == 0) {
938         // LED2 is GPIO94
939         GpioDataRegs.GPCT0GGLE.bit.GPIO94 = 1;
940     }
941 }
942
943 if (NewLVData == 1) {
944     NewLVData = 0;
945     printLV1 = fromLVvalues[0];
946     printLV2 = fromLVvalues[1];
947     /// = fromLVvalues[2];
948     /// = fromLVvalues[3];
949     /// = fromLVvalues[4];
950     /// = fromLVvalues[5];
951     /// = fromLVvalues[6];
952     /// = fromLVvalues[7];
953 }
954
955
956 if (new_optitrack == 1) {
957     OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
958     new_optitrack = 0;
959 }
960
961 // Step 0: update B, u
962 B[0][0] = cosf(ROBOTps.theta)*0.001;
963 B[1][0] = sinf(ROBOTps.theta)*0.001;
964 B[2][1] = 0.001;
965 u[0][0] = 0.5*(LeftVel + RightVel); // linear velocity of robot
966 u[1][0] = gyroz*(PI/180.0); // angular velocity in rad/s (negative for right hand angle)
967
968 // Step 1: predict the state and estimate covariance
969 Matrix3x2_Mult(B, u, Bu); // Bu = B*u
970 Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
971 Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
972 // Step 2: if there is a new measurement, then update the state
973 if (1 == newOPTITRACKpose) {
974     OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
975     if ((OptiNumUpdatesProcessed%10)==0) {
976         // LED 3
977         GpioDataRegs.GPCT0GGLE.bit.GPIO95 = 1;
978     }
979     newOPTITRACKpose = 0;
980     z[0][0] = OPTITRACKps.x; // take in the Optitrack Motion Capture measurement
981     z[1][0] = OPTITRACKps.y;
982     // fix for OptiTrack problem at 180 degrees
983     if (cosf(ROBOTps.theta) < -0.99) {
984         z[2][0] = ROBOTps.theta;
985     }
986     else {
987         z[2][0] = OPTITRACKps.theta;
988     }
989     // Step 2a: calculate the innovation/measurement residual, ytilde
990     Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0); // ytilde = z-x_pred
991     // Step 2b: calculate innovation covariance, S
992     Matrix3x3_Add(P_pred, R, S, 1.0, 1.0); // S = P_pred + R
993     // Step 2c: calculate the optimal Kalman gain, K
994     Matrix3x3_Invert(S, S_inv);
995     Matrix3x3_Mult(P_pred, S_inv, K); // K = P_pred*(S^-1)
996     // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
997     Matrix3x1_Mult(K, ytilde, temp_3x1);
998     Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
999     // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
1000     Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
1001     Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
1002 } // end of correction step
1003
1004 // set ROBOTps to the updated and corrected Kalman values.
1005 ROBOTps.x = x_pred[0][0];
1006 ROBOTps.y = x_pred[1][0];
1007 ROBOTps.theta = x_pred[2][0];
1008
1009 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1010 // uses xy code to step through an array of positions
1011 if( xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1012     statePos = (statePos+1)%NUMWAYPOINTS;
1013 }
1014 // state machine
1015 switch (RobotState) {
1016 case 1:
1017
1018     // vref and turn are the vref and turn returned from xy_control
1019
1020     if (LADARfront < 1.2) {
1021         vref = 0.2;
1022         checkfronttally++;
1023         if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
1024             RobotState = 10; // Wall follow

```

```

1025         WallFollowtime = 0;
1026         right_wall_follow_state = 1;
1027     }
1028 } else {
1029     checkfronttally = 0;
1030 }
1031
1032 break;
1033 case 10:
1034     if (right_wall_follow_state == 1) {
1035         //Left Turn
1036         turn = Kp_front_wall*(14.5 - LADARfront);
1037         vref = front_turn_velocity;
1038         if (LADARfront > left_turn_Stop_threshold) {
1039             right_wall_follow_state = 2;
1040         }
1041     } else if (right_wall_follow_state == 2) {
1042         //Right Wall Follow
1043         turn = Kp_right_wal*(ref_right_wall - LADARrightfront);
1044         vref = foward_velocity;
1045         if (LADARfront < left_turn_Start_threshold) {
1046             right_wall_follow_state = 1;
1047         }
1048     }
1049     if (turn > turn_saturation) {
1050         turn = turn_saturation;
1051     }
1052     if (turn < -turn_saturation) {
1053         turn = -turn_saturation;
1054     }
1055
1056     WallFollowtime++;
1057     if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1058         RobotState = 1;
1059         checkfronttally = 0;
1060     }
1061     break;
1062
1063 case 20:
1064     // put vision code here
1065     break;
1066 default:
1067     break;
1068 }
1069
1070
1071 //Must be called each time into this SWI1 function
1072 PIconrol(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1073 // These below lines also must be called each time into this SWI1 function
1074 setEPWM1A(uLeft);
1075 setEPWM2A(-uRight);
1076 setF28027EPWM1A(uLeft);
1077 setF28027EPWM2A(-uRight);
1078 LeftWheel_1 = LeftWheel;
1079 RightWheel_1 = RightWheel;
1080
1081 if((timecount%250) == 0) {
1082     DataToLabView.floatData[0] = ROBOTps.x;
1083     DataToLabView.floatData[1] = ROBOTps.y;
1084     DataToLabView.floatData[2] = ROBOTps.theta;
1085     DataToLabView.floatData[3] = (float)timecount;
1086     DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1087     DataToLabView.floatData[5] = (float)RobotState;
1088     DataToLabView.floatData[6] = (float)statePos;
1089     DataToLabView.floatData[7] = LADARfront;
1090     LVsenddata[0] = '*'; // header for LVdata
1091     LVsenddata[1] = '$';
1092     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1093         if (i%2==0) {
1094             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1095         } else {
1096             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1097         }
1098     }
1099     serial_sendSCID(&SerialD, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1100 }
1101
1102 }
1103 timecount++;
1104 if((timecount%200) == 0)
1105 {
1106     if(doneCal == 0) {
1107         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1108     }
1109     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1110     UARTPrint = 1; // Tell While loop to print
1111 }
1112
1113 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1114 CurrentChip = DAN28027;
1115 SpibRegs.SPIFRX.bit.RXFFIL = 3;
1116 GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
1117 SpibRegs.SPITXBUF = 0x00DA;
1118 SpibRegs.SPITXBUF = EPwm1A_F28027;
1119 SpibRegs.SPITXBUF = EPwm2A_F28027;
1120
1121
1122 //*****
1123 //
1124 // Restore registers saved:
1125 //
1126 DINT;
1127 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1128
1129 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
1130 }
1131
1132 //
1133 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1134 //
1135 interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1136 {
1137     // Set interrupt priority:
1138     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;

```

```

1139 IER |= M_INT12;
1140 IER &= MINT12; // Set "global" priority
1141 PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1142 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1143 __asm(" NOP");
1144 EINT;
1145
1146 //#####
1147 // Insert SWI ISR Code here.....
1148 // LED1
1149 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1150 if (LADARpingpong == 1) {
1151     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1152     LADARrightfront = 19; // 19 is greater than max feet
1153     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1154         if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1155             LADARrightfront = ladar_data[LADARi].distance_ping;
1156         }
1157     }
1158     // LADARfront is the min of dist 111, 112, 113, 114, 115
1159     LADARfront = 19;
1160     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1161         if (ladar_data[LADARi].distance_ping < LADARfront) {
1162             LADARfront = ladar_data[LADARi].distance_ping;
1163         }
1164     }
1165     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta));
1166     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)+LADARps.y*cosf(ROBOTps.theta));
1167     for (LADARi = 0; LADARi < 228; LADARi++) {
1168
1169         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1170         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1171     }
1172 } else if (LADARpingpong == 0) {
1173     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1174     LADARrightfront = 19; // 19 is greater than max feet
1175     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1176         if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1177             LADARrightfront = ladar_data[LADARi].distance_pong;
1178         }
1179     }
1180     // LADARfront is the min of dist 111, 112, 113, 114, 115
1181     LADARfront = 19;
1182     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1183         if (ladar_data[LADARi].distance_pong < LADARfront) {
1184             LADARfront = ladar_data[LADARi].distance_pong;
1185         }
1186     }
1187     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1188     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1189     for (LADARi = 0; LADARi < 228; LADARi++) {
1190
1191         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1192         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1193     }
1194 }
1195 }
1196 }
1197
1198
1199
1200
1201 //#####
1202 //
1203 // Restore registers saved:
1204 //
1205 DINT;
1206 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1207
1208 }
1209
1210 //
1211 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1212 //
1213 __interrupt void SWI3_LowestPriority(void) // FLASH_CORRECTABLE_ERROR
1214 {
1215     // Set interrupt priority:
1216     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1217     IER |= M_INT12;
1218     IER &= MINT12; // Set "global" priority
1219     PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1220     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1221     __asm(" NOP");
1222     EINT;
1223
1224     //#####
1225     // Insert SWI ISR Code here.....
1226
1227     RCangle = readEncWheel();
1228     setEPWM3A_RC Servo(RCangle); //RCangle is a value from -90 to 90
1229     setEPWM3B_RC Servo(RCangle); //RCangle is a value from -90 to 90
1230     setEPWM5A_RC Servo(RCangle); //RCangle is a value from -90 to 90
1231     setEPWM5B_RC Servo(RCangle); //RCangle is a value from -90 to 90
1232     setEPWM6A_RC Servo(RCangle); //RCangle is a value from -90 to 90
1233
1234     //#####
1235     //
1236     // Restore registers saved:
1237     //
1238     DINT;
1239     PieCtrlRegs.PIEIER12.all = TempPIEIER;
1240
1241 }
1242
1243 // ----- code for CAN start here -----
1244 __interrupt void can_isr(void)
1245 {
1246     int i = 0;
1247
1248     uint32_t status;
1249
1250     //
1251     // Read the CAN interrupt status to find the cause of the interrupt
1252     //

```

```

1253 status = CANGetInterruptCause(CANB_BASE);
1254
1255 //
1256 // If the cause is a controller status interrupt, then get the status
1257 //
1258 if(status == CAN_INT_INT0ID_STATUS)
1259 {
1260     //
1261     // Read the controller status. This will return a field of status
1262     // error bits that can indicate various errors. Error processing
1263     // is not done in this example for simplicity. Refer to the
1264     // API documentation for details about the error status bits.
1265     // The act of reading this status will clear the interrupt.
1266     //
1267     status = CANGetStatus(CANB_BASE);
1268 }
1269
1270 //
1271 // Check if the cause is the receive message object 2
1272 //
1273 else if(status == RX_MSG_OBJ_ID_1)
1274 {
1275     //
1276     // Get the received message
1277     //
1278     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1279
1280     for(i = 0; i<2; i++)
1281     {
1282         dis_raw_1[i] = rxMsgData[i];
1283     }
1284
1285     dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1286
1287     measure_status_1 = rxMsgData[2];
1288
1289     //
1290     // Getting to this point means that the RX interrupt occurred on
1291     // message object 2, and the message RX is complete. Clear the
1292     // message object interrupt.
1293     //
1294     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1295
1296     //
1297     // Increment a counter to keep track of how many messages have been
1298     // received. In a real application this could be used to set flags to
1299     // indicate when a message is received.
1300     //
1301     rxMsgCount_1++;
1302
1303     //
1304     // Since the message was received, clear any error flags.
1305     //
1306     errorFlag = 0;
1307 }
1308
1309 else if(status == RX_MSG_OBJ_ID_2)
1310 {
1311     //
1312     // Get the received message
1313     //
1314     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1315
1316     for(i = 0; i<2; i++)
1317     {
1318         dis_raw_2[i] = rxMsgData[i];
1319     }
1320
1321     dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1322
1323     measure_status_2 = rxMsgData[2];
1324
1325     //
1326     // Getting to this point means that the RX interrupt occurred on
1327     // message object 2, and the message RX is complete. Clear the
1328     // message object interrupt.
1329     //
1330     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1331
1332     //
1333     // Increment a counter to keep track of how many messages have been
1334     // received. In a real application this could be used to set flags to
1335     // indicate when a message is received.
1336     //
1337     rxMsgCount_2++;
1338
1339     //
1340     // Since the message was received, clear any error flags.
1341     //
1342     errorFlag = 0;
1343 }
1344
1345 else if(status == RX_MSG_OBJ_ID_3)
1346 {
1347     //
1348     // Get the received message
1349     //
1350     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1351
1352     for(i = 0; i<4; i++)
1353     {
1354         lightlevel_raw_1[i] = rxMsgData[i];
1355         quality_raw_1[i] = rxMsgData[i+4];
1356     }
1357
1358     lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1359     quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1360
1361     //
1362     // Getting to this point means that the RX interrupt occurred on
1363     // message object 2, and the message RX is complete. Clear the

```

```

1367 // message object interrupt.
1368 //
1369 CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1370
1371 //
1372 // Since the message was received, clear any error flags.
1373 //
1374 errorFlag = 0;
1375 }
1376
1377
1378 else if(status == RX_MSG_OBJ_ID_4)
1379 {
1380 //
1381 // Get the received message
1382 //
1383 CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1384
1385 for(i = 0; i<4; i++)
1386 {
1387     lightlevel_raw_2[i] = rxMsgData[i];
1388     quality_raw_2[i] = rxMsgData[i+4];
1389 }
1390
1391 lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1392 quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1393
1394 //
1395 // Getting to this point means that the RX interrupt occurred on
1396 // message object 2, and the message RX is complete. Clear the
1397 // message object interrupt.
1398 //
1399 CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1400
1401 //
1402 // Since the message was received, clear any error flags.
1403 //
1404 errorFlag = 0;
1405 }
1406
1407
1408
1409
1410 //
1411 // If something unexpected caused the interrupt, this would handle it.
1412 //
1413 else
1414 {
1415 //
1416 // Spurious interrupt handling can go here.
1417 //
1418 }
1419
1420 //
1421 // Clear the global interrupt flag for the CAN interrupt line
1422 //
1423 CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1424
1425 //
1426 // Acknowledge this interrupt located in group 9
1427 //
1428 InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1429 }
1430 // ----- code for CAN end here -----

```